



## PHASE 7

# Implementation Excerpts and Operations

Code excerpts, local setup and operational checks

SMARTPOCKET AI • SMART BUDGET AND RECOMMENDATION ASSISTANT

## 1. Selected Implementation Excerpts

The following excerpts are documentation-ready patterns aligned with the project material. They intentionally omit credentials and environment-specific values.

### FastAPI initialization and router registration

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routers import auth_router, recommend_router, history_router

app = FastAPI(title="SmartPocket AI")
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.frontend_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(auth_router.router)
app.include_router(recommend_router.router)
app.include_router(history_router.router)
```

### Protected recommendation route

```
@router.post("/recommend/party", response_model=RecommendationResponse)
async def recommend_party(
    payload: PartyRecommendRequest,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    candidate = await gemini_service.generate_party(payload)
    result = normalize_recommendation(candidate, payload.totalBudget)
    return result
```

### Environment-based configuration

```
GEMINI_API_KEY=your_key_here
JWT_SECRET_KEY=use_a_long_random_value
ACCESS_TOKEN_EXPIRE_MINUTES=1440
GEMINI_MODEL=your_supported_model
GEMINI_TIMEOUT_SECONDS=45
FRONTEND_ORIGINS=http://localhost:5500
```



## Frontend authenticated request

```
const token = localStorage.getItem("access_token");
const response = await fetch(`${API_BASE}/recommend/home`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${token}`
  },
  body: JSON.stringify(payload)
});
if (!response.ok) throw new Error(await response.text());
const plan = await response.json();
```

## 2. Setup and Operations

### 2.1 Local Setup

| Step | Action                                           | Verification                                                                       |
|------|--------------------------------------------------|------------------------------------------------------------------------------------|
| 1    | Install Python and create a virtual environment. | The selected interpreter points to the project virtual environment.                |
| 2    | Install packages from requirements.txt.          | FastAPI, Uvicorn, SQLAlchemy, authentication, and AI packages import successfully. |
| 3    | Create .env using non-public values.             | The service reads configuration without printing secrets.                          |
| 4    | Start the API with Uvicorn.                      | The health route and interactive API docs load.                                    |
| 5    | Serve the Frontend directory.                    | The landing page opens from the configured frontend origin.                        |
| 6    | Register or use an approved test account.        | Login returns a valid token and the dashboard loads.                               |
| 7    | Run each planner with a small test budget.       | Results remain within budget and can be saved to history.                          |

### Backend command

```
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

### Simple frontend server

```
cd Frontend
python -m http.server 5500
```

### 2.2 Operational Checks

- Confirm the frontend origin matches the backend CORS configuration.
- Confirm the model identifier is supported by the configured AI SDK.
- Confirm the database file is writable by the running process.
- Confirm expired or invalid tokens return 401 without exposing internal details.
- Confirm AI failures do not create partial history entries.
- Confirm logs do not contain authorization headers, passwords, or API keys.